

EEG Speed Toolkit

Composable API guide for EEG-only reach-speed classification

A standalone function library for building your own training function. No high-level call owns your experiment, filesystem, split loop, or reporting.

Object	What it contains
Package	eeg_speed_toolkit
Primary target	Fast versus slow reach completion within participant and grasp task
Primary EEG feature	Time-resolved 0-5 Hz movement-related cortical potential
Recommended validation	Leave one subject out, with group-disjoint inner tuning
Python	3.10 or newer
Branch	feature/eeg-speed-toolkit

Version 0.1 - 6 October 2026

1. Design contract

The toolkit is intentionally assembled from small operations. A function either validates, constructs labels, returns indices, creates features, fits one model, or calculates metrics. No public function combines all of these steps.

The application is responsible for the outer loop. This makes the train, validation, and test boundaries visible in your own code and prevents a short wrapper from hiding leakage-sensitive work.

Recommended call order

Object	What it contains
1. Configuration	load_config() or default_config(), then validate_config().
2. Dataset	load_dataset() or construct EEGSpeedDataset, then validate_dataset().
3. Target	Choose calibrated within-subject labels or outer-training normative labels.
4. Split	Request index arrays. Inspect them before touching a model.
5. Features	Create one EEG matrix. Fit any learned transform on training rows only.
6. Model	Construct or tune one estimator with caller-supplied training arrays.
7. Prediction	Call estimator.predict() on validation or test rows.
8. Evaluation	Pass predictions and labels to metric and permutation functions.

What the library never does

It never creates a run directory, chooses an outer fold, saves a model, loads raw participant recordings, or calls a monolithic train_experiment routine. Persistence is intentionally outside the package.

2. Dataset and configuration

EEGSpeedDataset

A frozen dataclass whose first array axis is always trial. Required fields:

Object	What it contains
mr_cp_full	trials x channels x samples, normally -500 to +300 ms around onset
mr_cp_pre	trials x channels x samples, normally -500 to 0 ms
erd	trials x fixed ERD features
duration	positive reach duration in seconds
reaction_time	nonnegative cue-to-onset time in seconds
tasks / subjects / trials	one string identifier per trial; trial identifiers must be unique
mr_cp_frontal / posterior	optional spatial-control epochs
manifest	optional metadata, including motor channel names

load_dataset(path) -> EEGSpeedDataset

Purpose. Load the documented NPZ contract. It performs no fitting or filtering.

Use it when. You already produced eeg_speed_features_v2.npz or another compatible artifact.

validate_dataset(dataset, config=None) -> DatasetSummary

Purpose. Check trial counts, ranks, finite values, positive durations, unique identifiers, and feature availability.

Use it when. Immediately after constructing or loading a dataset, before labels or splits.

Important. Validation is read-only. Passing config adds checks for the selected feature.

subset_dataset(dataset, indices) -> EEGSpeedDataset

Purpose. Create an immutable trial subset across every array.

Use it when. You need a compact dataset object for inspection or a downstream component.

Important. For model fitting, direct matrix indexing is usually clearer.

3. Configuration functions

`default_config()` -> dict

Purpose. Return a new mutable copy of the recommended configuration.

Use it when. You construct configuration in Python and want safe defaults.

`load_config(path)` -> dict

Purpose. Read YAML, validate it, and return a normalized copy.

Use it when. Configuration belongs in a versioned YAML file.

`validate_config(config)` -> dict

Purpose. Validate supported feature, target, split, model, and evaluation options without mutating the input.

Use it when. Your code creates or modifies a mapping before use.

Supported top-level sections

Object	What it contains
seed	Integer used only where the caller passes the configuration onward.
feature	name plus include_context flag. The flag documents intent; you still assemble context explicitly.
target	within_subject_task_median or training_task_median.
split	mode, excluded subjects, and inner group-fold count.
model	logistic model, C candidates, class weighting, iterations, and worker count.
evaluation	permutation repeat count.

```
config = default_config()
config["feature"]["name"] = "mr_cp_laplacian"
config["split"]["exclude_subjects"] = ["14"]
config = validate_config(config)
```

4. Target construction

`make_within_subject_task_labels(duration, tasks, subjects) -> TargetLabels`

Purpose. Create 0=fast and 1=slow labels relative to each participant-and-task median.

Use it when. Your scientific question concerns trial-to-trial speed differences after removing person and grasp baselines.

Important. This target uses the held-out participant's outcome distribution. It is calibrated, not zero-calibration deployment.

`make_training_task_median_labels(duration, tasks, train_indices, test_indices) -> FoldTargets`

Purpose. Learn one median per task from outer-training outcomes and apply those thresholds to train and test rows.

Use it when. You want a normative unseen-participant target with no threshold information from the test subject.

Important. Call it separately inside every outer fold.

Returned target objects

Object	What it contains
TargetLabels.labels	One binary label per input trial.
TargetLabels.thresholds	Nested subject then task threshold mapping.
FoldTargets.train_labels	Labels aligned to train_indices, not the complete dataset.
FoldTargets.test_labels	Labels aligned to test_indices.
description	Human-readable definition suitable for a report.

Do not calculate one global median before subject-held-out testing. That mixes participant and task effects and can create deceptively strong pooled performance.

5. Splitting functions

leave_one_subject_out_splits(subjects, exclude_subjects=()) -> iterator[IndexSplit]

Purpose. Yield explicit outer train/test indices, once per retained subject.

Use it when. You need subject-independent evaluation.

grouped_validation_splits(groups, candidate_indices=None, n_splits=5)

Purpose. Yield train/validation indices with no group overlap.

Use it when. You are writing custom hyperparameter selection and want inner subject-disjoint folds.

Important. Pass only the outer-training indices as candidate_indices.

subject_partition(subjects, validation_subjects, test_subjects=(), exclude_subjects=())

Purpose. Create one caller-specified subject-disjoint train/validation/test partition.

Use it when. You want a fixed, reviewable split rather than cross-validation.

personalized_contiguous_splits(tasks, trial_numbers, n_splits=5)

Purpose. Hold out one contiguous trial chunk from every task.

Use it when. You train one personalized model and need temporal rather than random generalization.

IndexSplit

Each split contains train, validation, optional test, and a name. Empty validation arrays in LOSO are deliberate: outer LOSO defines train/test, while you explicitly choose an inner validation method.

6. EEG preprocessing helpers

`average_reference(eeg) -> ndarray`

Purpose. Subtract the channel average from 2D continuous EEG or 3D epochs.

Use it when. Your input has not already been average-referenced.

`extract_epoch(signal, event_time_s, window_s, sampling_rate_hz)`

Purpose. Extract a copied channels x samples interval around an event.

Use it when. You are building the feature dataset from continuous or trial-wise EEG.

`lowpass_filter(eeg, sampling_rate_hz, cutoff_hz=5.0)`

Purpose. Apply the fourth-order zero-phase low-pass used for offline MRCP extraction.

Use it when. You want MRCP morphology rather than oscillatory EEG.

Important. Zero-phase filtering is offline and noncausal. Do not describe it as a real-time filter.

`baseline_correct(epoch, baseline) -> ndarray`

Purpose. Subtract each channel's baseline mean.

Use it when. You have separately extracted a pre-cue baseline for the same trial.

`resample_epoch(epoch, original_hz, target_hz) -> ndarray`

Purpose. Polyphase-resample the final time axis.

Use it when. You want a compact fixed-rate epoch, such as 50 Hz MRCP samples.

```
referenced = average_reference(trial_eeg)
baseline = extract_epoch(referenced, cue_s, (-0.9, -0.5), fs)
epoch = extract_epoch(referenced, movement_onset_s, (-0.5, 0.3), fs)
mr_cp = lowpass_filter(epoch, fs, cutoff_hz=5.0)
mr_cp = baseline_correct(mr_cp, lowpass_filter(baseline, fs, 5.0))
mr_cp_50hz = resample_epoch(mr_cp, int(fs), 50)
```

7. Feature assembly

`flatten_epochs(epochs) -> ndarray`

Purpose. Convert trials x channels x samples into trials x features.

Use it when. You want a conventional estimator input while preserving all channel-time values.

`central_surface_laplacian(epochs, channel_names=None) -> ndarray`

Purpose. Subtract local FC, CP, and lateral neighbors from seven central channels.

Use it when. You want to suppress spatially global movement or reference activity.

Important. The input must contain the documented 21-channel motor grid.

`make_eeg_feature_matrix(dataset, name) -> ndarray`

Purpose. Select and flatten mrpc_full, mrpc_pre, mrpc_laplacian, spatial controls, or ERD.

Use it when. You want a validated fixed feature representation with no learned parameters.

`ContextEncoder.fit / transform / fit_transform`

Purpose. One-hot encode task and append reaction time using training-known task categories.

Use it when. You are building an explicit non-EEG control or additive baseline.

Important. Fit on outer-training tasks, then transform validation/test tasks.

`append_context(eeg_features, context_features) -> ndarray`

Purpose. Concatenate two prepared matrices by trial.

Use it when. You explicitly choose to test incremental EEG value beyond context.

The recommended primary EEG feature is mrpc_full. Always report mrpc_pre and mrpc_laplacian as controls, because the observed signal emerged after movement onset and was spatially broad.

8. Training-only spatial filtering

`epoch_covariances(epochs) -> ndarray`

Purpose. Calculate trace-normalized covariance for every epoch.

Use it when. You are implementing spatial covariance features or inspecting CSP inputs.

`FilterBankCSP(sampling_rate_hz, bands, filters_per_side, regularization)`

Purpose. Create a binary filter-bank common spatial pattern transformer.

Use it when. You want learned mu/beta spatial filters as an alternative to fixed MRCP features.

`FilterBankCSP.fit(train_epochs, train_labels)`

Purpose. Learn band-specific generalized eigenvectors from training classes only.

Use it when. Inside an outer split, before touching validation or test epochs.

`FilterBankCSP.transform(epochs) -> ndarray`

Purpose. Apply already learned spatial filters and return log-variance features.

Use it when. After fit, for training, validation, or test epochs.

```
csp = FilterBankCSP(sampling_rate_hz=250)
x_train = csp.fit_transform(raw_epochs[train], y[train])
x_test = csp.transform(raw_epochs[test])
```

Never call `fit_transform` separately on test data. That would estimate class-sensitive spatial filters from the held-out participant.

9. Model functions

make_logistic_classifier(config, c=None, seed=None) -> Pipeline

Purpose. Construct an unfitted StandardScaler plus class-weighted logistic classifier.

Use it when. You already chose C or want a single explicit model.

fit_classifier(estimator, train_features, train_labels) -> estimator

Purpose. Clone and fit one estimator on exactly the supplied rows.

Use it when. You manage validation yourself or fit a final model after selecting settings.

Important. The input estimator is not modified.

tune_logistic_classifier(train_features, train_labels, train_groups, config)

Purpose. Select C with group-disjoint inner CV and refit on all supplied outer-training rows.

Use it when. You want the recommended nested subject-held-out procedure.

Important. The function cannot access outer test arrays because they are not arguments.

ModelSelectionResult

Object	What it contains
estimator	Fitted scaler-plus-logistic pipeline, refit on all supplied training rows.
best_c	Selected inverse regularization strength.
validation_score	Mean inner balanced accuracy for best C.
cv_results	Full scikit-learn model-selection diagnostics.

Call `estimator.predict(x)` and `estimator.predict_proba(x)` directly. Thin wrappers would hide the standard estimator interface without adding safety.

10. Evaluation functions

classification_metrics(true_labels, predicted_labels) -> dict

Purpose. Return balanced accuracy, macro F1, confusion matrix, and sample count.

Use it when. After you have predictions for one fold or pooled held-out predictions.

per_subject_metrics(true_labels, predicted_labels, subjects) -> dict

Purpose. Calculate the same metrics for every participant.

Use it when. You need to show whether a pooled result is consistent across people.

permutation_pvalue(true_labels, predicted_labels, subjects, tasks, repeats, seed)

Purpose. Shuffle labels within participant and task while keeping predictions fixed.

Use it when. You need a null that preserves participant and grasp composition.

Important. Use pooled outer-held-out predictions, not training predictions.

Balanced accuracy is primary because fold-specific class proportions can differ even when labels are median-based. Always retain the confusion matrix and participant table.

11. Complete caller-owned assembly

The following example is intentionally longer than three lines. Every leakage-sensitive decision is visible in the application function.

```
import numpy as np
from eeg_speed_toolkit import (
    load_dataset, load_config, validate_dataset,
    make_eeg_feature_matrix, make_within_subject_task_labels,
    leave_one_subject_out_splits, tune_logistic_classifier,
    classification_metrics, per_subject_metrics, permutation_pvalue,
)

def my_training_function(dataset, config):
    validate_dataset(dataset, config)
    x = make_eeg_feature_matrix(dataset, config["feature"]["name"])
    target = make_within_subject_task_labels(
        dataset.duration, dataset.tasks, dataset.subjects
    )

    prediction = np.full(dataset.n_samples, -1, dtype=int)
    evaluated = np.zeros(dataset.n_samples, dtype=bool)
    models = {}

    splits = leave_one_subject_out_splits(
        dataset.subjects,
        exclude_subjects=config["split"]["exclude_subjects"],
    )
    for split in splits:
        selection = tune_logistic_classifier(
            x[split.train],
            target.labels[split.train],
            dataset.subjects[split.train],
            config,
        )
        prediction[split.test] = selection.estimator.predict(x[split.test])
        evaluated[split.test] = True
        models[split.name] = selection
```

11. Complete caller-owned assembly - continued

```

truth = target.labels[evaluated]
predicted = prediction[evaluated]
result = classification_metrics(truth, predicted)
result["subjects"] = per_subject_metrics(
    truth, predicted, dataset.subjects[evaluated]
)
result["permutation_p"] = permutation_pvalue(
    truth,
    predicted,
    dataset.subjects[evaluated],
    dataset.tasks[evaluated],
    repeats=config["evaluation"]["permutations"],
    seed=config["seed"],
)
return models, prediction, result

config = load_config("config/example.yaml")
dataset = load_dataset("eeg_speed_features_v2.npz")
models, prediction, result = my_training_function(dataset, config)

```

Your application may replace the model, feature matrix, split iterator, result structure, and persistence. The toolkit functions remain useful because their contracts are independent.

Using normative labels instead

```

for split in leave_one_subject_out_splits(dataset.subjects, ["14"]):
    target = make_training_task_median_labels(
        dataset.duration, dataset.tasks, split.train, split.test
    )
    selection = tune_logistic_classifier(
        x[split.train], target.train_labels,
        dataset.subjects[split.train], config,
    )
    prediction[split.test] = selection.estimator.predict(x[split.test])

```

12. Leakage and interpretation checklist

Object	What it contains
Dataset	All arrays share the same trial order; trial IDs are unique; units and event definitions are documented.
Target	State whether labels use held-out participant calibration or outer-training normative thresholds.
Outer split	A participant never appears in both train and test for subject-independent claims.
Inner split	Regularization and learned spatial filters use only outer-training participants.
Scaling	StandardScaler remains inside the estimator pipeline and is fit on training rows.
CSP	fit() sees training epochs and labels only; transform() handles held-out epochs.
Context	Reaction time and task are reported as controls, not silently concatenated.
Metrics	Use pooled held-out balanced accuracy plus per-subject results.
Null	Shuffle labels within participant and task.
Claim	The successful MRCP window includes early execution. Do not call it pre-movement intention decoding.
Spatial caveat	Frontal/posterior controls were strong; retain the central-Laplacian analysis.

Recommended minimum report

Report feature window, onset source, target calibration, excluded participants, outer and inner splits, chosen C distribution, pooled balanced accuracy, macro F1, confusion matrix, participant scores, permutation p-value, pre-onset control, frontal/posterior controls, and Laplacian control.

13. Quick reference

Object	What it contains
config.py	default_config, load_config, validate_config
dataset.py	EEGSpeedDataset, DatasetSummary, load_dataset, subset_dataset, validate_dataset
targets.py	TargetLabels, FoldTargets, calibrated and training-only median labels
splitting.py	IndexSplit, LOSO, grouped inner folds, fixed partition, contiguous personalized folds
preprocessing.py	average reference, epoch extraction, low-pass, baseline correction, resampling
features.py	flattening, central Laplacian, EEG selector, context encoder, explicit concatenation
spatial.py	epoch covariance and training-only FilterBankCSP
models.py	logistic pipeline construction, direct fitting, group-aware C selection
evaluation.py	pooled metrics, participant metrics, stratified permutation p-value

Core principle: the toolkit supplies testable building blocks; your function remains the visible owner of the experiment.

Project layout

```
eeg-speed-toolkit/
  config/example.yaml
  examples/manual_subject_holdout.py
  src/eeg_speed_toolkit/
    config.py      dataset.py      targets.py
    splitting.py   preprocessing.py features.py
    spatial.py     models.py       evaluation.py
  tests/test_toolkit.py
  output/pdf/eeg_speed_toolkit_guide.pdf
```